

A Local Image Service Is Not a Chat Model: Reproducible Qwen-Image 2.1 Deployment on Apple Silicon

Chengyi Xu and KLIK team
KLIK Research

September 2026 — engineering reproducibility note

Abstract

We document a local deployment of the requested Qwen-Image 2.1 Uncensored GGUF checkpoint on an Apple M4 Max with 64 GiB unified memory. The system uses a Q6_K diffusion checkpoint, a BF16 Qwen3VL8B text encoder, the version-specific BF16 VAE, and a pinned Metal build of stable-diffusion.cpp. A genuine Model Context Protocol (MCP) tool invokes the native image-generation API rather than pretending that an image model is a text provider. A 512×512, 20-step, seed-42 request completes through the registered MCP path in 373.74 seconds; the returned 236,756-byte PNG passes decoding, dimensional and nonblank checks, and local OCR recovers the requested word “LOCAL.” These checks are not a semantic or aesthetic image-quality evaluation. We verify exclusive model switching, a generation-held controller lock, and shutdown. This note reports one integration image, not a quality benchmark or a text-token throughput claim. The model’s research/evaluation-only license restriction is retained and weights are not redistributed.

1 Objective and modality boundary

A practical local stack may require normal text generation, a separate abilitated text checkpoint, and image generation. These are not interchangeable capabilities. A 262,144-token text context limit and a 20-token/s text-stream target say nothing about image generation. Registering an image checkpoint as a chat model creates false expectations about tokenization, tool calling and API semantics.

Our image deployment therefore exposes a native image API and two MCP tools: `local_image_status` and `generate_local_image`. Text models remain separate providers. The Desktop controller admits one large inference engine at a time; selecting image mode stops the text services only after their current qualification work has ended. Cloud-provider selections are preserved.

2 Pinned model and runtime

The requested source is

abenzerps/Qwen-Image-2.1-Uncensored-GGUF [1].

Its pinned revision is 8fa47c0eb8da2323cd756f01404601ed4ff4706e. The deployment uses qwen-image-2.1-UC-Q6_K.gguf, qwen3vl_8b_bf16.safetensors, and qwen_image_2.1_vae_bf16.safetensors. Component sizes and hashes were verified after Hugging Face CLI downloads; a successful download command alone was not accepted as integrity evidence.

The runtime is stable-diffusion.cpp at commit 2f886889e6e8b78738d6b87f7191f6018557c551, built in Release mode with Metal and pinned submodules [2]. It serves on loopback with the native

model identifier `sd-cpp-local`. CPU offload and flash attention are enabled. The host is an M4 Max with 40 GPU cores and 64 GiB unified memory, macOS 27 build 26A428. The machine is concurrently used; no quiet-host performance assumption is made.

The applicable Qwen Research License restricts use to research/evaluation without separate permission for commercial use [3]. The checkpoint’s “uncensored” label is not treated as a license grant. We publish runtime integration code and evidence, not model weights, and do not relicense third-party runtimes or checkpoints.

3 Request and control path

The verified request path is

registered stdio MCP → local adapter → native images API → validated PNG.

The test reads the actual CC Switch MCP registration, initializes the server, lists tools, and checks `local_image_status` before requesting generation. It does not substitute a directly called Python function for the MCP exchange.

The adapter restricts dimensions to multiples of 32 in 256–1024, step count to 1–50, and seed to a nonnegative 31-bit integer. Native extra arguments are serialized by the adapter; prompt-embedded native-option tags are rejected. Requests use `/v1/images/generations`, a single image, base64 JSON delivery and PNG output. The adapter validates base64, bounds response size, decodes the PNG, and saves it under a generated-image directory. There is no cloud fallback.

During generation the adapter holds the same exclusive lock used by the model controller. A nonmutating acquisition probe confirmed that another switch could not acquire the lock. We did not deliberately interrupt a generation to test destructive recovery. The controller verifies model identity/readiness before declaring success and refuses conflicting lab-engine ports.

4 Measured request and result

The prompt requests a red cube beside a blue sphere on a white table, with a white card reading “LOCAL,” neutral lighting and a simple composition. The request specifies 512×512, 20 steps and seed 42. The standard image profile uses Euler sampling and CFG 6. The request receipt records these supplied parameters; the inspected native log independently confirms dimensions, Euler and generation completion time. The excerpt does not independently attest every parameter.

Measurement	Result
Adapter generation elapsed	373.737s
Including MCP startup	374.074s
Native logged generation time	373.66s
Native logged latent-image stage	196.45s
Output dimensions	512×512
PNG size	236,756bytes
Red-pixel fraction (heuristic)	9.39%
Blue-pixel fraction (heuristic)	5.72%
OCR text	LOCAL

Table 1: One actual MCP/API generation. These are integration observations, not population estimates.

The PNG SHA-256 is

d76a733009ca383e63e2358d305375431193efe844b344d54bcbea3ea4f96872.

Decoded RGB standard deviations are 48.12, 60.66 and 58.82, providing a simple nonblank check. Local Apple Vision OCR returns “LOCAL” with its reported confidence 1.0. That number is an API score, not a calibrated probability that the entire requested scene is correct. Red/blue pixel thresholds are inexpensive checks of color presence, not shape recognition.

An earlier direct-CLI 512×512 smoke took approximately 737 seconds. It differed in path and run conditions. Comparing that observation with 374 seconds does not establish an MCP speedup: an adapter is not an acceleration algorithm, and host/cache conditions were not controlled.

5 What was and was not verified

Verified. Pinned component integrity; native startup; correct model identity; registered MCP initialization and tool listing; actual image API generation; valid nonblank PNG; requested word recovered by OCR; exclusive controller lock during generation; text-to-image switching; final shutdown with all deployment ports closed.

Not verified. Expert visual quality, exact cube/sphere geometry, typography beyond the OCR observation, broad prompt adherence, content-policy/refusal characteristics, deterministic pixel reproduction across runtime revisions, concurrent serving capacity, commercial suitability or comprehensive recovery after a native crash. The reviewing chat model could not view image attachments, and no human visual review is invented.

6 Reproduction and practical use

Use the pinned runtime and model revision, verify all downloaded component hashes, and start image mode through the on-demand controller. Invoke the registered MCP tool with the request parameters recorded in the artifact. Preserve the returned path, actual elapsed time, native log, PNG dimensions and checksum. Keep model licensing with the installation. Stop the service before starting another large inference runtime.

Image performance should be reported as time to image with dimensions, steps, sampler and hardware context. Text-only 256k-context and tokens/s labels must not be attached to this endpoint. Readiness proves that a service is reachable, not that a particular image is aesthetically successful.

7 Conclusion and disclosure

The measured local integration works through the complete registered MCP/native-API path, not merely a command-line model smoke. It remains a small engineering verification with explicit visual-quality and performance limitations. More images, repeated controlled runs and independent visual assessment are needed for a quality benchmark.

This note and its implementation are AI-assisted. No independent human review, peer acceptance, funding statement or conflict attestation is asserted. Public hosting is not journal acceptance. Attribution is Chengyi Xu and KLIK team as requested by the project owner. Model and runtime licenses remain separate from the artifact code license.

References

- [1] Requested checkpoint. <https://huggingface.co/abenzerps/Qwen-Image-2.1-Uncensored-GGUF>.
- [2] stable-diffusion.cpp. <https://github.com/leejet/stable-diffusion.cpp>.
- [3] Qwen-Image2.1 license. <https://huggingface.co/Qwen/Qwen-Image-2.1/blob/main/LICENSE>.